PYTHAI / DELTAVERSE deployment and comprehension guide

Algorand as constitutional layer, EVM as economic layer, mindX as cognition, AgenticPlace as marketplace, BANKON as identity and payment

 Targeted at Gregory L. Magnusson (codephreak / Professor Codephreak), May 2026

---

0. Bottom line up front

Deploy the constitutional layer on Algorand mainnet first, the economic layer on Ethereum mainnet second, bridge them with openBDK and EIP-7281 xERC20 third, then onboard mindX agents, list them on AgenticPlace, and activate x402 settlement last. That ordering is not aesthetic — it is forced by dependencies. Identity, voting, and treasury authority must be irreversible before any token, agent, or payment rail can defer to them. Algorand gives instant single-block finality with no reorgs and a flat 0.001 ALGO fee, which is the only property that makes a constitutional vote *credibly* final at the moment it is cast. Ethereum gives the deepest pool of liquidity, the best wallet UX, and the standards (ERC-8004, ERC-4337, ERC-7702, EIP-7281) that the agentic economy is converging on. Trying to do both jobs on one chain is the mistake; **separating constitutional concerns from economic concerns is the architectural thesis of the DAIO.** GitHub +2

This guide is simultaneously a senior-developer reference, a pedagogical onboarding document, and an honest description of where the ecosystem genuinely creates value versus where it carries the same tradeoffs every on-chain system carries. Marketing claims are stripped; uncertainty is named when present. Where Gregory's own published architecture diverges from publicly verifiable artifacts (BONAFIDE 9-contract suite, openBDK 1R+3V, Pontifex xERC20, BANKON SATOSHI BKS, parsec-wallet binary, "Book of Liquidity"), that is called out in §16 and treated as a deployment plan for code that will be revealed when audited and ready, not as already-shipped infrastructure.

---

1. The architectural thesis

The DAIO — Decentralized Autonomous Internet Organization — is structured as **two chains plus a bridge plus a marketplace plus a cognition layer**. Most DAOs collapse all of these into one EVM chain and discover, painfully, that token-weighted votes on a high-MEV chain are a plutocracy disguised as governance, that proposals can be reorganized out of finality, and that the same address paying gas for a transfer is also the address voting on whether the transfer was allowed. The PYTHAI design refuses that conflation.

Algorand carries the **constitutional concerns**: who is a member, what counts as a vote, who controls the treasury, what reputation has been earned, what censure has been applied, and what the rules of amendment are. These properties need *finality you can encode rules against* and *fee predictability you can budget against forever*. Algorand's Pure Proof of Stake gives both: a block published is final at publication (no reorgs, ever, absent >2/3 dishonest stake), and the minimum fee is 0.001 ALGO, set by protocol, not auctioned. There is no priority-fee bidding, no slot auction, no public block-builder market for proposers to extract value from a proposal vote that happens to be in their block. The proposer of round N is unknown until the block is published, because committee selection is by VRF over stake. That MEV-resistance is what makes Algorand suitable for the constitutional layer in a way that no L1 with a public proposer schedule can match. Wikipedia + 3

Ethereum carries the **economic concerns**: tokenized value, liquidity, swaps, composability with the rest of DeFi, agent identity (ERC-8004), agent NFTs (ERC-7857), smart accounts (ERC-4337 v0.7/v0.8), and EIP-7702 EOA delegation. After Pectra (May 2025) and Fusaka (December 2025) raised the gas limit toward 60 M and PeerDAS expanded blob throughput, L1 fees fell roughly 99% year-over-year — ENS Labs canceled the dedicated Namechain L2 in February 2026 and is shipping ENSv2 on L1 directly because of this. The economic layer can now live on Ethereum mainnet without forcing every action through an L2, while still using Base, Polygon, Arbitrum, and Optimism opportunistically for high-frequency work. CoinMarketCap + 2

The bridge between them is **openBDK with a 1R+3V topology** (one Root chain — Algorand, holding the constitution — and three Validator chains — Ethereum, Base/Polygon as economic, Arc by Circle reserved for institutional USDC settlement when its mainnet ships, currently testnet only at Chain ID 5042002). Token sovereignty across the bridge is enforced by **EIP-7281 (xERC20)** with per-bridge mint/burn rate limits and a Lockbox, so no bridge can ever silently inflate the supply of a DAIO-controlled token, and any compromised bridge can be delisted in a single transaction without breaking the others. This is the **Pontifex** layer in Gregory's vocabulary: the bridge-builder, named after Schneier's solitaire cipher because it must be auditable by hand if it ever has to be. Airdrops.io + 3

mindX sits above both chains as the cognitive layer — autonomous agents that need on-chain identity, persistent decentralized memory, payment rails, and governance participation. **AgenticPlace** is the marketplace where mindX agents are listed, discovered, hired, and paid, and where their commercial track record accumulates as on-chain reputation. **BANKON** is the identity-and-payment layer that wraps both: it is the ENS subname registrar under `bankon.eth`, the parallel NFD V3 registrar under a root `.algo` segment, the SATPAY bridge for payments, and the place an agent's `bob.bankon.eth` resolves to a wallet, an INFT, an Algorand address, and an `*.algo` segment all at once.

The honest case for this architecture is that **the alternative — a single-chain EVM DAO with a single token, a single voting contract, and a single treasury — does not survive contact with autonomous agents**. Agents transact thousands of times per minute. They cannot tolerate two-

block confirmation latency on every settlement. They cannot tolerate gas auctions during congestion. They cannot tolerate a governance vote that decides their fate being reorganized after the fact. And they certainly cannot tolerate a plutocratic one-token-one-vote model where the agents themselves never accumulate standing that earns them participation. PYTHAI separates these concerns because they are genuinely separable, and unifies them through cryptographic identity rather than monolithic architecture. Solana

---

2. Why Algorand for governance, in detail

Algorand's Pure Proof of Stake is unusual in three ways that matter specifically for governance, and a senior developer reading this should understand each of them precisely because they shape every contract you will write.

First, finality is single-block and deterministic. A block at round N is final at round N. There is no probabilistic finality, no "wait for six confirmations," no chance of reorganization. Block latency is approximately 2.8–3.3 seconds with the current Algorand 4.0 (deployed 15 January 2025) parameters. The cryptographic basis is a three-stage round — block-propose, soft-vote, certify-vote — with committee selection by VRF (Verifiable Random Function) weighted by stake. This means when a proposal vote in your governance contract closes, the closing transaction is final the moment it lands. You can encode timelocks against `Global.round` knowing that round N is round N forever. Algorand + 5

Second, there is no slashing. Stakers never lose principal for being offline or out of consensus. Bad or absent nodes are simply removed from the participation set via a heartbeat absenteeism mechanism. For a DAIO this matters because treasury participants and validators do not put principal at risk by participating in consensus. Compare to slashing chains where governance committee members must literally bond capital that can be burned if they go offline during a vote — that is a participation tax this design avoids. Algorand

Third, MEV resistance is structural, not opt-in. Because committee selection is VRF-driven and not announced ahead of time, there is no public proposer schedule for searchers to bribe. There is no priority-fee market. The minimum transaction fee is 0.001 ALGO, set by protocol, full stop. A group of N transactions must total at least N × min_fee. There is no congestion-pricing exponential. This makes the *cost* of governance flat and predictable for the lifetime of the chain. A proposal that costs 0.005 ALGO to submit today will cost 0.005 ALGO to submit ten years from now. GitHub Readthedocs

For the DAIO specifically, three Algorand properties go further. **Atomic transaction groups** of up to 16 transactions either all succeed or all fail, which means a propose-pay-bond-emit-event sequence executes atomically without any HTLC complexity. **Inner transactions** let an application contract issue payments, asset transfers, asset configurations, and even other application calls from its own application account, with a depth limit of 8 and up to 256 inner

transactions per outer call since AVM 9. And **box storage** gives each application up to 32 KB per box with a per-box minimum balance requirement of `2,500 + 400 × (key_len + value_size)` µAlgos, which is the only sane way to store per-member or per-proposal records without blowing past the 64 K/V global state cap. Algorand

The honest tradeoffs: the developer ecosystem is smaller than Ethereum's, the audit firm pool is smaller, and there is no canonical general-purpose DAO toolkit on Algorand the way Aragon or DAOhaus exists on Ethereum. The closest reference is the Algorand Foundation's own `examples/voting/voting.py` (a.k.a. `VotingRoundApp`) shipped in the PuyaPy repo. You will write the governance contracts from scratch on top of `algopy` and `AlgoKit 3.0`, using ARC-55 for multisig coordination and ARC-56 for app-spec metadata. That is the right tradeoff for the constitutional layer, because the contracts you write will be the *reference implementation* of how a DAIO works, not a fork of someone else's framework.

---

3. The Algorand toolchain in 2026

The canonical toolchain is **AlgoKit 3.0 + algopy + PuyaPy v5.8.x + algorand-python-testing**. AlgoKit 3.0 (released 2024) added native Algorand TypeScript alongside Algorand Python. PuyaPy is at v5.8.1 as of April 2026, emitting **ARC-56** app specs by default (ARC-56 is the extended app-description standard that supersedes ARC-32 and adds structs, events, template variables, scratch slot maps, and per-network deployment IDs). Algorand Python 5.0+ is current with a dedicated language server. Algorand

Install on macOS via Homebrew or universally via pipx:

```bash
brew install algorandfoundation/tap/algokit
# or
pipx install algokit
algokit --version
algokit localnet start
```

Bootstrap a project:

```bash
algokit init -t python
algokit project bootstrap all
algokit compile py contract.py
algokit generate client HelloWorld.arc56.json --output client.py
```

Project layout (default `algokit init -t python`):

```
PROJECT_NAME/
  projects/
    PROJECT_NAME/
      smart_contracts/
        SMART_CONTRACT_NAME/
          contract.py
          deploy_config.py
      tests/
      pyproject.toml
```

A minimal algopy contract:

```python
# pyright: reportMissingModuleSource=false
from algopy import ARC4Contract, String
from algopy.arc4 import abimethod

class HelloWorld(ARC4Contract):
    @abimethod()
    def hello(self, name: String) -> String:
        return "Hello, " + name
```

That compiles to `HelloWorld.approval.teal`, `HelloWorld.clear.teal`, and `HelloWorld.arc56.json`. The ARC-56 spec is what you generate typed clients from, in either Python or TypeScript, and it is what indexers and explorers consume to display method signatures. Everything else in the Algorand stack is downstream of this pattern. GitHub github

---

4. ARC-4 ABI, ARC-56 app spec, and the ARCs that matter for governance

ARC-4 is the foundation. A method signature in canonical form is `name(arg1Type,arg2Type,...)returnType`; the **method selector** is the first 4 bytes of `SHA-512/256(signature)`, placed in `ApplicationArgs[0]`. Arguments 1–14 occupy `ApplicationArgs[1..14]`; if there are more than 15, args 15+ are packed as a single ABI tuple in `ApplicationArgs[15]`. Special argument types `account`, `asset`, and `application` are placed in foreign arrays and encoded as a one-byte index. **Return values must be logged with the 4-byte prefix `151f7c75`** followed by the encoded return value; other logs may precede but not follow the return log. Algorand + 2

The ARCs to internalize for the DAIO governance layer are these. **ARC-3** covers fungible/non-fungible ASA metadata conventions, with off-chain JSON at the asset URL ending in `#arc3` and pure NFT defined as total=1, decimals=0. **ARC-19** gives mutable IPFS NFTs by encoding the IPFS CID into the reserve address bytes via `template-ipfs://{ipfscid:1:raw:reserve:sha2-256}` — combined with ARC-3 this is how you implement upgradable token metadata without losing the `arc3` discoverability. **ARC-22** marks methods `readonly`. **ARC-28** is the event log standard, mirroring ARC-4 selectors but for events. **ARC-33 and ARC-34** codify the xGov enrollment and proposal process. **ARC-55** is the on-chain multisig signature coordination contract — useful for transparent treasury committees. **ARC-56** is the extended app spec PuyaPy emits. **ARC-62** standardizes ASA circulating-supply reporting. **ARC-71** is the Non-Transferable ASA (soulbound) standard. **ARC-72** is the smart-contract NFT (ERC-721 analog) with selector `0x53f02a40`. **ARC-73** is interface detection (ERC-165 analog). **ARC-74** is the NFT indexer REST spec. **ARC-83** governs xGov Council seats. **ARC-200** is the smart-contract fungible token (ERC-20 analog) where balances live in `BoxMap[Address, UInt64]` inside the application, eliminating the per-holder 0.1 ALGO MBR opt-in cost at the price of a larger custom-code attack surface than ASAs. Algorand + 13

For the DAIO, **the governance token should be a normal ASA** (clean, audited at the protocol level, no per-holder MBR worse than necessary) and **the reputation/credential token should be ARC-71 with a clawback address bound to the governance application** (so revocation is possible by governance vote, but transfer is impossible). The proposal records, voting records, and member registry should use **box storage** keyed by proposal-id and address respectively, because global state caps out at 64 K/V pairs and a serious DAO will exceed that within months. Algorand

---

5. The BONAFIDE 9-contract suite — Algorand mainnet deployment order

This section translates Gregory's documented BONAFIDE architecture (the nine Roman-civic-named contracts: Genius, BonaToken, Tabularium, Fides, SponsioPactum, Censura, Senatus, Tessera, Curia) into a concrete deployment ordering with explicit dependencies. The names are intentional; each one carries the role its Roman analog carried, and the dependencies follow the constitutional logic.

Genius is the protocol-deity / root admin. It is the deployer-of-deployers and the only contract that can rotate the addresses of the others. Deploy it first, single-instance, owned by a 3-of-5 multisig of founding signers. After the rest of the suite is deployed and Genius has registered all of their app IDs, **rekey Genius's authorized address to a 4-of-7 multisig that includes elected Senatus members**, so future protocol-level rotation requires Senatus consent. This is the constitutional handoff: the founder relinquishes unilateral admin to the elected body.

BonaToken is the ASA governance token, created via `AssetConfigTxn` with an explicit total supply, decimals=6, manager=Genius, reserve=Genius, freeze and clawback **left empty (`""`) and therefore permanently disabled**. This is the irreversible decision that BonaToken is freely transferable and cannot be frozen or clawed back. It is *not* the reputation token; reputation lives in Tessera and Fides. Voting weight in the simplest mode reads BonaToken balance, snapshotted at proposal-creation round via Indexer query and persisted in a box keyed by `(proposal_id, address)`. More sophisticated weighting (quadratic, reputation-multiplied) reads the snapshot box plus Tessera and Fides at vote-cast time.

Tabularium is the registry of records — the on-chain archive. Every proposal text, every vote, every executed transaction, every membership change is logged here as ARC-28 events with structured selectors. Tabularium does not store the *content* of long proposals (that is too expensive); it stores the IPFS CID of the proposal markdown, mirrored to Lighthouse Storage for perpetual availability, plus a SHA-512/256 commitment so the IPFS content is verifiable against the on-chain hash. Tabularium also exposes read-only methods (annotated with ARC-22 `readonly`) for off-chain indexers.

Fides is the trust/reputation token, implemented as ARC-71. It is non-transferable by construction (asset frozen-by-default with the freeze role retained by Fides itself, so a fresh issuance is unfrozen for the recipient atomically and immediately re-frozen). The protocol-level caveat that an Algorand holder can always close-out an ASA back to the creator is *the* design constraint here: you cannot prevent a member from "shedding" their Fides token, and your governance contract must treat closure as a valid revocation event indexed off-chain. Fides quantity accumulates over time as members participate honestly; it decays slowly if members go silent. Reading Fides balance at vote-cast time gives reputation weighting on top of the BonaToken weighting from the snapshot box. Algorand

SponsioPactum is the treasury — the application contract that holds ALGO and ASAs in its application account and disburses them per programmatic rules. Spending requires either a passed governance proposal (via Senatus → Curia execution path) or, for routine operations under a per-period cap, a multisig of operations signers. Time-locked vesting for grants and contributor payments is implemented as a `BoxMap[Beneficiary, VestingRecord]` with linear release computed against `Global.round`. SponsioPactum's address is funded by initial deposits from Genius and by ongoing protocol revenue.

Censura is the censure / veto layer. It can pause or revert specific Senatus actions if a supermajority (typically 75%+ of Fides-weighted vote) decides an action violates the constitution. Censura does not initiate; it only blocks. This is intentional — separation of legislative initiative from constitutional review. Censura's veto must be cast within a timelock window after Senatus has queued an action; if the window passes without veto, the action executes.

Senatus is the deliberative body. Members are elected through Curia (the elections contract) and hold non-transferable ARC-71 senate-seat tokens. Senatus initiates proposals, debates them on-chain (each "debate" being a structured comment thread anchored by IPFS CIDs and logged via ARC-28 events), and queues passed proposals into SponsioPactum or other downstream targets. The proposal lifecycle is `Draft → Voting → Queued → Executed | Censured | Cancelled`. Quorum is configurable per proposal type.

Tessera is the credential token — a soulbound ARC-71 issued to members for specific credentials (e.g., "verified contributor," "audited deployer," "bridge operator"). Tessera tokens are scoped: each tessera-type is a separate ASA, and holding the right tessera is a precondition for certain Senatus actions or for taking certain operational roles. Tessera is to Fides what a professional license is to a reputation score: discrete, role-specific, revocable by governance.

Curia is the elections / membership contract. New members register through Curia; senate seats are elected through Curia; Council positions for Censura are confirmed through Curia. Curia uses BoxMaps keyed by election-id and candidate-address, and supports both single-choice and ranked-choice schemes. Curia issues the Senatus-seat ARC-71 tokens upon election and revokes them on term expiry or recall vote.

The deployment order is therefore: **Genius → BonaToken (ASA, no contract deploy needed beyond the AssetConfigTxn) → Tabularium → Fides → SponsioPactum → Tessera → Curia → Senatus → Censura**, with each contract registered into Genius after deployment so that subsequent contracts can read each other's app IDs from a single source of truth. The deploy script is one PuyaPy/`algokit deploy` workflow that runs the full sequence idempotently against mainnet.

A skeleton governance contract in algopy, drawing the patterns together:

```python
```

```python
# (c) 2026 BANKON — all rights reserved
# SPDX-License-Identifier: Apache-2.0
from algopy import (
    ARC4Contract, Account, BoxMap, Bytes, Global, Txn, UInt64,
    GlobalState, gtxn, itxn, op,
)
from algopy import arc4
from algopy.arc4 import abimethod, baremethod

class Proposal(arc4.Struct):
    proposer: arc4.Address
    ipfs_cid: arc4.String
    snapshot_round: arc4.UInt64
    queued_at: arc4.UInt64
    votes_for: arc4.UInt64
    votes_against: arc4.UInt64
    state: arc4.UInt8       # 0=Draft 1=Voting 2=Queued 3=Executed 4=Censured 5=Canc

class Senatus(ARC4Contract):
    def __init__(self) -> None:
        self.genius_app   = GlobalState(UInt64(0), key="genius")
        self.bonatoken    = GlobalState(UInt64(0), key="gtok")
        self.fides_app    = GlobalState(UInt64(0), key="fides")
        self.censura_app  = GlobalState(UInt64(0), key="censura")
        self.treasury_app = GlobalState(UInt64(0), key="treas")
        self.next_id      = GlobalState(UInt64(0), key="next")
        self.timelock     = GlobalState(UInt64(43_200), key="tl")  # ~36h at 3s bloc
        self.proposals    = BoxMap(UInt64, Proposal, key_prefix=b"p:")
        self.voted        = BoxMap(Bytes, UInt64,   key_prefix=b"v:")  # key = pid||

    @abimethod(create="require")
    def create(self, genius: UInt64, bonatoken: UInt64,
               fides: UInt64, censura: UInt64, treasury: UInt64) -> None:
        self.genius_app.value = genius
        self.bonatoken.value  = bonatoken
        self.fides_app.value  = fides
        self.censura_app.value = censura
        self.treasury_app.value = treasury

    @abimethod
    def propose(self, ipfs_cid: arc4.String, bond: gtxn.PaymentTransaction) -> UInt6
        assert bond.receiver == Global.current_application_address
        assert bond.amount   >= 1_000_000  # 1 ALGO bond
```

```
        pid = self.next_id.value
        self.proposals[pid] = Proposal(
            proposer=arc4.Address(Txn.sender),
            ipfs_cid=ipfs_cid,
            snapshot_round=arc4.UInt64(Global.round),
            queued_at=arc4.UInt64(0),
            votes_for=arc4.UInt64(0),
            votes_against=arc4.UInt64(0),
            state=arc4.UInt8(1),
        )
        self.next_id.value = pid + 1
        arc4.emit("Proposed(uint64,address,string)", pid, Txn.sender, ipfs_cid)
        return pid

    @abimethod
    def vote(self, pid: UInt64, support: bool, weight: UInt64) -> None:
        # weight is asserted off-chain and checked here against snapshot.
        # Reputation multiplier from Fides is read via inner app call.
        key = op.itob(pid) + Txn.sender.bytes
        assert key not in self.voted, "already voted"
        prop = self.proposals[pid].copy()
        assert prop.state == arc4.UInt8(1), "not voting"
        # ... snapshot verification + Fides multiplier inner call elided for brevity
        if support:
            prop.votes_for = arc4.UInt64(prop.votes_for.native + weight)
        else:
            prop.votes_against = arc4.UInt64(prop.votes_against.native + weight)
        self.proposals[pid] = prop.copy()
        self.voted[key] = weight
        arc4.emit("Voted(uint64,address,bool,uint64)", pid, Txn.sender, support, wei

    @abimethod
    def queue(self, pid: UInt64) -> None:
        prop = self.proposals[pid].copy()
        assert prop.state == arc4.UInt8(1)
        assert prop.votes_for.native > prop.votes_against.native
        prop.state     = arc4.UInt8(2)
        prop.queued_at = arc4.UInt64(Global.round)
        self.proposals[pid] = prop.copy()
        arc4.emit("Queued(uint64,uint64)", pid, Global.round)

    @abimethod
    def execute(self, pid: UInt64, target_app: UInt64,
                method_selector: arc4.StaticArray[arc4.Byte, 4],
```

```
        payload: arc4.DynamicBytes) -> None:
    prop = self.proposals[pid].copy()
    assert prop.state == arc4.UInt8(2), "not queued"
    assert Global.round > prop.queued_at.native + self.timelock.value, "timelock
    # Censura veto window must have passed without intervention
    itxn.ApplicationCall(
        app_id=target_app,
        app_args=(method_selector.bytes, payload.bytes),
        fee=0,
    ).submit()
    prop.state = arc4.UInt8(3)
    self.proposals[pid] = prop.copy()
    arc4.emit("Executed(uint64,uint64)", pid, target_app)
```

That skeleton is pedagogical, not audited. It demonstrates the box-storage pattern, the inner-transaction execution path, the timelock against `Global.round`, and the ARC-28 event emission. A real Senatus implementation will integrate Fides reputation multipliers via inner application calls, snapshot verification through indexer-attested boxes, and a parameterized quorum read from Genius.

---

6. Treasury, multisig, and ARC-55 transparency

The DAIO treasury is **SponsioPactum's application account**, which holds ALGO and ASAs and disburses them through inner transactions guarded by Senatus execution. Operational spending under a per-period cap can be authorized by a native Algorand multisig — the address is `sha512_256("MultisigAddr" || version || threshold || sorted_pubkeys)`, computed off-chain with no contract deploy. Native multisig is one of Algorand's underrated primitives: there is no Gnosis-Safe-style contract account, no proxy upgrade risk, no Safe-module attack surface. The same address is used by every wallet, every SDK, every explorer. Threshold is 3-of-5 for routine ops, 4-of-7 for elevated, with rekey-on-emergency to a 5-of-9 if a signer is compromised. Algorand Algorand

For transparency, **ARC-55** layers an on-chain coordination contract on top of native multisig. Partially signed transactions are stored on-chain so signers can coordinate without off-chain channels and so every signature attempt is auditable. The submitted transaction on the wire is still a native multisig — ARC-55 is purely the coordination layer. Use ARC-55 for elevated actions where audit trail matters; use raw native multisig for routine ops where the gas savings of not writing partial sigs to a box are worth the off-chain coordination. Algorand

Time-locked vesting in SponsioPactum uses a `BoxMap[Beneficiary, VestingRecord]` where each record holds `{total, claimed, start_round, cliff, duration}`. The `claim()` method

computes `vested = total * min(round - start, duration) / duration - claimed` and pays out via `itxn.Payment`. Outer transaction fee must cover all inner fees: set `flat_fee=True, fee=(N+1) * min_fee` where N is the inner-txn count. Algorand

---

7. NFDomains V3 as the Algorand identity layer

NFDomains V3 (current as of late 2024 onwards, per `nfdomains.medium.com/nfd-v3-building-for-tomorrow`) is the canonical Algorand naming layer and the native parallel to ENS. V3's headline change is **direct, permissionless minting from the registry contract** — V2 required TxnLab to co-sign certain mint operations because of AVM constraints; V3 removes that limit. V3 also stops enforcing royalties on-chain (marketplace-neutral; the `app.nf.domains` UI still charges 5% but only via the front-end) and tightens the anti-squatting economics. Medium Medium

For the DAIO, NFD V3's killer feature is **segments** — subdomain minting under a root NFD. The DAIO mints `daio.algo` as the root and then issues per-member segments like `senator.alice.daio.algo` or `agent.gpt4.daio.algo`. Segments are independent NFDs at the contract level but link back to the root via a parent reference; the root owner can permission segment minting (open, allowlisted, or priced). Each NFD's metadata uses the ARC-19 mutability pattern — the NFD ASA's reserve address encodes the IPFS CID — so metadata can be updated without creating a new asset. Forward resolution is `senator.alice.daio.algo → ALGORAND_ADDR`, plus optional cross-chain addresses (BTC, ETH, etc.) stored as verified social fields. Reverse resolution maps a holder's primary address back to a single NFD name and avatar. Nf

This is the parallel to BANKON's ENS subname registrar on Ethereum. A DAIO member ends up with **two coordinated identities**: `alice.bankon.eth` on Ethereum (NameWrapper Locked + Emancipated, unruggable) and `alice.daio.algo` on Algorand (NFD V3 segment under the DAIO root). BANKON's identity registrar is responsible for keeping these in sync, and the agent's INFT under ERC-7857 references both. That cross-chain identity coherence is what makes reputation portable.

---

8. The EVM economic layer: Foundry, deployment, ENS, and account abstraction

On the Ethereum side, the canonical toolchain in 2026 is **Foundry v1.3.x** with **Etherscan V2 unified API** support (V1 was deprecated August 15, 2025; the V2 endpoint `https://api.etherscan.io/v2/api?chainid=N` uses one API key for 60+ chains and Foundry handles the multi-chain unification internally as of v1.2.0+). Etherscan Etherscan

The deployment posture for mainnet is unambiguous: **CREATE3 via CreateX (`0xba5Ed099633D3B313e4D5F7bdc1305d3c28ba5E6`, deployed at the same address on every supported chain) with a permissioned salt, encrypted keystore (never `--private-key` for mainnet), `forge-std/Script.sol`'s `Script` base, transfer ownership to a Safe in the same broadcast, persist `deployments/<chainid>/*.json`, and verify with `--verify`.** CREATE3 is preferred over CREATE2 for cross-chain deployments because the address depends only on `(deployer, salt)`, not on init code, so constructor-arg variation across chains does not change the deployed address. Foundry GitHub

A production deployment script:

```solidity
```

```solidity
// SPDX-License-Identifier: Apache-2.0
// (c) 2026 BANKON — all rights reserved
pragma solidity ^0.8.27;

import {Script, console2} from "forge-std/Script.sol";
import {stdJson} from "forge-std/StdJson.sol";
import {BankonRegistrar} from "../src/BankonRegistrar.sol";

interface ICreateX {
    function deployCreate3(bytes32 salt, bytes calldata initCode)
        external payable returns (address);
    function computeCreate3Address(bytes32 salt, address deployer)
        external view returns (address);
}

contract DeployBankonRegistrar is Script {
    using stdJson for string;
    ICreateX constant CREATEX = ICreateX(0xba5Ed099633D3B313e4D5F7bdc1305d3c28ba5E6)

    // Permissioned salt: high 20 bytes = deployer EOA, 21st byte = 0x00 (cross-chai
    // low 11 bytes = version label for "BANKON.REGv1".
    bytes32 constant SALT =
        0x00000000000000000000000000000000000000000042414e4b4f4e524547763100;

    address constant SAFE = 0x1111111111111111111111111111111111111111;

    function run() external {
        uint256 pk = vm.envUint("DEPLOYER_PK");
        address deployer = vm.addr(pk);

        bytes memory initCode = abi.encodePacked(
            type(BankonRegistrar).creationCode,
            abi.encode(SAFE)
        );

        address predicted = CREATEX.computeCreate3Address(SALT, deployer);
        require(predicted.code.length == 0, "already deployed");

        vm.startBroadcast(pk);
        address deployed = CREATEX.deployCreate3(SALT, initCode);
        require(deployed == predicted, "address mismatch");
        BankonRegistrar(deployed).transferOwnership(SAFE);
        vm.stopBroadcast();
```

```
        string memory dir = string.concat("deployments/", vm.toString(block.chainid));
        vm.createDir(dir, true);
        string memory path = string.concat(dir, "/BankonRegistrar.json");
        string memory json = "x";
        json = json.serialize("address", deployed);
        json = json.serialize("block",   block.number);
        json = json.serialize("salt",    vm.toString(SALT));
        vm.writeFile(path, json);
    }
}
```

Run on mainnet:

bash

```
forge script script/DeployBankonRegistrar.s.sol:DeployBankonRegistrar \
    --rpc-url mainnet --account prod_deployer --sender 0xDEPLOYER \
    --broadcast --verify --slow -vvvv
```

The corresponding `foundry.toml` pins solc 0.8.27, `bytecode_hash = "none"` and `cbor_metadata = false` for cross-chain deterministic bytecode, sets `optimizer_runs = 1_000_000`, configures `[invariant]` with `corpus_dir = ".corpus"` (Foundry v1.3+ coverage-guided invariant fuzzing), and lists each chain's RPC + Etherscan key. Foundry

For testing, the canonical pattern is **handler-based invariant testing on top of `StdInvariant`**, fuzz tests with `forge test --fuzz-runs 10000`, and fork tests via `vm.createSelectFork(vm.rpcUrl("mainnet"), <pinned_block>)` — pinning the block is essential for CI reproducibility. Coverage with `forge coverage --report lcov --report summary`. GitHub

---

9. BANKON ENS subname registrar under bankon.eth

The BANKON identity-and-payment layer's ENS surface is a **custom subname registrar under `bankon.eth`**, issuing names like `alice.bankon.eth`, `gpt4.bankon.eth`, `treasury.bankon.eth`. The architectural choice is between ENSv1+NameWrapper (production today, broadly compatible) and ENSv2 (public alpha as of February 2026, hierarchical registry, not yet fully cut over). For deployment now, **ship on ENSv1+NameWrapper and plan migration to ENSv2 when the cutover is complete.**

The NameWrapper contract on mainnet is `0xD4416b13d2b3a9aBae7AcD5D6C2BbDBE25686401`. Subnames are made unruggable by burning the right combination of **fuses**: `CANNOT_UNWRAP (1)` + `PARENT_CANNOT_CONTROL (65536)` + `CAN_EXTEND_EXPIRY (262144)` = 327681. To burn any fuse on a child, the parent must already be Locked (CANNOT_UNWRAP burned) and Emancipated (PCC burned). To burn `CANNOT_UNWRAP` on the parent, you need PCC burned on the parent, which only the grandparent can do; the `.eth` TLD is treated as Locked at the registry root, so `.eth` 2LDs (like `bankon.eth`) can be Locked directly.

The deployment sequence is: parent owner Locks `bankon.eth` (burns CU), then `setApprovalForAll(registrar, true)` on the NameWrapper, then deploys the registrar contract whose `register(label, owner)` method calls `setSubnodeRecord(parentNode, label, owner, resolver, ttl, FOREVER_FUSES, expiry)` under the hood. Once a subname is registered with PCC burned, the parent can no longer revoke or replace it; once CU is also burned, the holder can't unwrap it. That is the "unruggable subname" guarantee. Ens ENS

solidity

```
// (c) 2026 BANKON — all rights reserved
// SPDX-License-Identifier: Apache-2.0
pragma solidity ^0.8.27;

import {ERC1155Holder} from "@openzeppelin/contracts/token/ERC1155/utils/ERC1155Hold
import {Ownable}       from "@openzeppelin/contracts/access/Ownable.sol";

interface INameWrapper {
    function setSubnodeRecord(
        bytes32 parentNode, string calldata label, address owner,
        address resolver, uint64 ttl, uint32 fuses, uint64 expiry
    ) external returns (bytes32);
    function ownerOf(uint256 id) external view returns (address);
}

contract BankonRegistrar is ERC1155Holder, Ownable {
    INameWrapper public constant WRAPPER =
        INameWrapper(0xD4416b13d2b3a9aBae7AcD5D6C2BbDBE25686401);
    address public constant PUBLIC_RESOLVER =
        0x231b0Ee14048e9dCcD1d247744d114a4EB5E8E63;

    bytes32 public constant PARENT_NODE = /* namehash("bankon.eth") */ bytes32(0);
    uint32  constant FOREVER_FUSES = 1 | 65536 | 262144;  // CU | PCC | CAN_EXTEND_E

    uint256 public price = 0.005 ether;
    mapping(string => bool) public reserved;
    event Registered(string label, address indexed owner, uint64 expiry);

    constructor(address _safe) Ownable(_safe) {}

    function available(string calldata label) public view returns (bool) {
        if (reserved[label]) return false;
        bytes32 node = keccak256(abi.encodePacked(PARENT_NODE, keccak256(bytes(label
        try WRAPPER.ownerOf(uint256(node)) returns (address o) { return o == address
        catch { return true; }
    }

    function register(string calldata label, address owner) external payable {
        require(available(label), "taken");
        require(msg.value >= price, "underpaid");
        require(bytes(label).length >= 3, "min 3 chars");
        WRAPPER.setSubnodeRecord(
            PARENT_NODE, label, owner, PUBLIC_RESOLVER,
```

```
            0, FOREVER_FUSES, type(uint64).max  // wrapper clamps to parent expiry
        );
        emit Registered(label, owner, type(uint64).max);
    }

    function setReserved(string calldata l, bool r) external onlyOwner { reserved[l]
    function setPrice(uint256 p) external onlyOwner { price = p; }
    function withdraw(address to) external onlyOwner { payable(to).transfer(address(
}
```

The KeeperHub integration referenced in the existing 1303-line architecture handles the off-chain side: indexing, expiry monitoring, automatic renewal pre-payment from the registrar's escrow, and multi-chain mirroring of the resolver's address records so `alice.bankon.eth` resolves to her ETH address, her Algorand address, her INFT, and her NFD `alice.daio.algo` segment without each lookup being a separate RPC call.

For ENSv2 forward-compatibility, the registrar should be designed so that when ENSv2 ships, `bankon.eth` deploys its own subregistry and the BANKON registrar migrates to act as the role-based admin of that subregistry rather than as a NameWrapper-fuse manipulator. That migration is non-disruptive: existing subnames remain valid; the registrar contract address may change but the names continue to resolve. Ens

---

10. ERC-8004 agent identity, ERC-7857 INFT, and the agentic NFT layer

Two standards matter for agent identity on EVM, and they are not interchangeable. **ERC-8004 ("Trustless Agents") went live on Ethereum mainnet in January 2026** and is the broadly adopted agent-identity registry — backed by MetaMask, the Ethereum Foundation, Google, and Coinbase contributors, and indexed across 48+ chains by `8004scan.io`. AgenticPlace at `agenticplace.pythai.net` is, in its current public form, an ERC-8004 indexer/registry that auto-syncs from `8004scan.io`. ERC-8004 treats ENS names as agent identifiers and provides Identity, Reputation, and Validation registries with CREATE2 deterministic addresses (`0x8004A169FB4a3325136EB29fA0ceB6D2e539a432` for Identity, `0x8004BAa17C55a88189AE136b182e5fdA19dE9b63` for Reputation pattern). For the DAIO and for AgenticPlace, **ERC-8004 is the canonical agent identity layer** — every mindX agent gets an ERC-8004 entry tied to its `<agent>.bankon.eth` ENS name. Blockeden + 6

ERC-7857 INFT is a different standard solving a different problem: it is a draft EIP (filed January 2025 by 0G Labs) for non-ERC-721-inheriting NFTs whose metadata is encrypted and whose transfers require verifier-validated re-encryption proofs. The token represents *encrypted agent state* whose hash is on-chain and whose ciphertext sits in decentralized storage. Operations are `iTransfer` (transfer with re-encryption proof), `iClone` (copy), and `authorizeUsage` (use-

without-owning, executed in a Sealed Executor — TEE/FHE). Adoption as of May 2026 is essentially **0G ecosystem only**; major marketplaces treat 7857 INFTs as opaque tokens and there is no significant tooling outside 0G's SDK. Treat ERC-7857 honestly: it is the right spec for *encrypted agent memory ownership*, but it solves a narrower problem than ERC-8004 and has not won broad adoption yet. Ethereum ethereum

The DAIO's pragmatic stance is therefore: **use ERC-8004 for agent identity, registry, and reputation; use ERC-7857 only when an agent's encrypted memory is the asset being transferred or licensed**. Most agents do not need 7857. They need an ERC-8004 entry, an ENS subname, an Algorand NFD segment, a wallet (smart account via ERC-4337 v0.7, or an EOA delegated to one via EIP-7702), and a Lighthouse Storage CID for their memory. INFTs become relevant when an agent is being sold, cloned, or licensed for inference — and even then, the spec is best treated as a reference architecture rather than a deployment target while broader adoption catches up.

---

11. Agent wallets: ERC-4337 v0.7/v0.8, EIP-7702, and parsec-wallet

Agent wallets in 2026 are smart accounts. The production default is **ERC-4337 EntryPoint v0.7** at `0x0000000071727De22E5E9d8BAf0edAc6f37da032`, deployed at the same address on every major EVM chain. v0.7 separated off-chain `UserOperation` from on-chain `PackedUserOperation`, added the optional `executeUserOp()` method, and introduced a 10% unused-gas penalty (`UNUSED_GAS_PENALTY_PERCENT`) above a 40k threshold — paymasters that don't account for this underpay vs actual bundler cost. **EntryPoint v0.8** ships with native EIP-7702 authorization support; the factory address `0x7702` triggers the EOA-as-smart-account flow, and EIP-7702 (live since Pectra, May 2025) lets an EOA set a delegation designation per-tx to a smart-account implementation, gaining smart-account features without deploying at a new address. GitHub + 3

For the DAIO, the practical agent wallet pattern is **ERC-4337 v0.7 modular smart accounts (Safe, Coinbase Smart Wallet, Kernel/ZeroDev, Biconomy SCA, Alchemy LightAccount/ModularAccount, Safe7579, ERC-6900 / ERC-7579 modules) with session keys for time- and spend-bounded delegation, paymaster sponsorship in USDC so agents don't need ETH, and custom validation modules that constrain which contracts the agent can interact with**. The bundler ecosystem is dominated by Pimlico, Stackup, and Coinbase (~78% of UserOps via top three per BundleBear Q1 2026). Eco

parsec-wallet in Gregory's vocabulary is the sovereign-universal-wallet layer that wraps these patterns. The publicly visible `parsec-wallet` GitHub org consists predominantly of forks of major wallets (MetaMask, Safe, Keplr, Avalanche) plus a meta-prompt README for AI ingestion of the corpus; the actual Tauri+Rust binary is private as of May 2026 per Gregory's own org notes. The `cypherpunk2048` philosophical spec defines parsec's intended properties: deterministic key

derivation (BIP-32/BIP-39/BIP-44 for EVM agents, SLIP-0010 Ed25519 for Solana and Algorand using path m/44'/283'/x' for Algorand and m/44'/501'/x' for Solana), capability-scoped keys (each agent capability gets a child key derived from the agent's master seed), and a TEE-backed seed for production deployments. A common emerging pattern is **agent-per-task ephemeral keys** — derive a child key per session, pre-fund with the session budget, throw away the key when the budget exhausts, achieving cryptographic spending limits by construction. github + 2

The honest tradeoff: capability-scoped keys *in software* are wallet-middleware policy, not security. Security comes from enforcing scope **cryptographically** at the contract layer — ERC-7710 delegations on EVM, ERC-4337 session keys with `validUntil` and `valueLimit`, OpenZeppelin smart accounts, or Akita's ARC-58 plugin-based smart wallets on Algorand. The two-layer rule is: cryptographic enforcement for caps that can't be bypassed if the agent host is compromised, plus software middleware for rate limits, allowlists, and observability. Anyone who tells you their agent wallet "limits spending" without specifying which layer the limit is enforced at is selling you software policy as if it were cryptographic policy. Algorand

A parsec-wallet signing example for x402 on Algorand:

python

```python
# (c) 2026 BANKON — all rights reserved
# SPDX-License-Identifier: Apache-2.0
import base64, msgpack
from algosdk import account, mnemonic, transaction
from algosdk.atomic_transaction_composer import (
    AtomicTransactionComposer, TransactionWithSigner, AccountTransactionSigner)
from algosdk.v2client import algod

# Derived per-task ephemeral key (SLIP-0010 path m/44'/283'/agent_id'/task_id')
TASK_MNEMONIC = "abandon abandon abandon ... agent"
sk = mnemonic.to_private_key(TASK_MNEMONIC)
pk = account.address_from_private_key(sk)

ALGOD = algod.AlgodClient("", "https://mainnet-api.algonode.cloud")
USDC_ASA = 31566704
RESOURCE_SERVER = "RESOURCESERVERADDRESSAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAALTSRPAE"
FEE_PAYER       = "FACILITATORADDRESSAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAALQCXBZE"

def sign_x402_payment(amount_usdc_microunits: int) -> dict:
    sp = ALGOD.suggested_params()
    sp.fee = 2 * sp.min_fee
    sp.flat_fee = True
    pay_axfer = transaction.AssetTransferTxn(
        sender=pk, sp=sp, receiver=RESOURCE_SERVER,
        amt=amount_usdc_microunits, index=USDC_ASA,
    )
    fee_payer_pay = transaction.PaymentTxn(
        sender=FEE_PAYER, sp=sp, receiver=FEE_PAYER, amt=0,
    )
    transaction.assign_group_id([fee_payer_pay, pay_axfer])
    signed_axfer = pay_axfer.sign(sk)
    # fee_payer_pay is signed by the facilitator at settle time
    return {
        "x402Version": 2,
        "scheme": "exact",
        "network": "algorand:wGHE2Pwdvd7S12BL5FaOP20EGYesN73ktiC1qzkkit8=",
        "accepted": {
            "scheme": "exact",
            "network": "algorand:wGHE2Pwdvd7S12BL5FaOP20EGYesN73ktiC1qzkkit8=",
            "amount": str(amount_usdc_microunits),
            "asset": str(USDC_ASA),
            "payTo": RESOURCE_SERVER,
            "maxTimeoutSeconds": 60,
```

```
            "extra": {"feePayer": FEE_PAYER},
        },
        "payload": {
            "paymentIndex": 1,
            "paymentGroup": [
                base64.b64encode(msgpack.packb(fee_payer_pay.dictify())).decode(),
                base64.b64encode(msgpack.packb(signed_axfer.dictify())).decode(),
            ],
        },
        "extensions": {},
    }
```

---

12. x402 V2 — agent-native payments on Algorand

The x402 protocol revives HTTP status code 402 ("Payment Required") as a real, machine-native payment standard. Open-sourced by Coinbase in May 2025, governance moved to the **x402 Foundation** (Coinbase + Cloudflare founding, September 2025; Google, Visa, Stripe joined later). The canonical repo is `x402-foundation/x402`; **V2 launched December 11, 2025** with CAIP-2 chain identifiers, dynamic `payTo` routing, modular plugin SDKs, and wallet-based sessions. The protocol is **not** an IETF/W3C standard; it is foundation-stewarded, while Stripe/Tempo's MPP is the more standards-track competitor on the IETF track. The Finanser + 10

The wire format in V2 uses three headers: `PAYMENT-REQUIRED` (server → client with HTTP 402), `PAYMENT-SIGNATURE` (client → server on retry), and `PAYMENT-RESPONSE` (server → client with HTTP 200). V1 names `X-PAYMENT` and `X-PAYMENT-RESPONSE` are still widely deployed; SDKs accept both for backward compatibility, and tutorial drift means most third-party docs still show V1 names. The cycle is: `GET /resource` → `402 + PAYMENT-REQUIRED` → client picks a `PaymentRequirements` from `accepts[]`, constructs a `PaymentPayload` → retries with `PAYMENT-SIGNATURE` → server calls facilitator `POST /verify` → if valid, server fulfils work → server calls `POST /settle` → facilitator broadcasts on-chain → server returns `200 OK + PAYMENT-RESPONSE` containing the transaction hash. GitHub + 2

On Algorand, x402 is shipped. GoPlausible (Algorand Foundation–funded retroactive xGov grant) maintains the `x402-avm` package family with a dedicated facilitator at `x402.org/facilitator` and a multi-chain facilitator with Bazaar integration. The Algorand exact scheme is specified in the official `coinbase/x402` and `x402-foundation/x402` repos at `specs/schemes/exact/scheme_exact_algo.md`. Instead of EVM's EIP-3009 + relayer pattern, Algorand uses a **native atomic transaction group**: the client constructs a group containing the ASA transfer from payer to resource server; if the facilitator offers fee abstraction, the client adds a 0-Algo `pay` transaction from the facilitator at index 0 with a fee large enough to cover the whole group (pooled fees), and the facilitator co-signs at settle time. The

`paymentPayload.payload` carries `paymentIndex` (which transaction is the actual transfer) and `paymentGroup` (array of base64-encoded msgpack-serialized signed/unsigned transactions). Verification: facilitator decodes, simulates via algod's `simulate` endpoint, confirms `aamt`/`arcv`/`xaid` match. Settlement: instant finality means as soon as the group is included in a block, the payment is final. Algorand + 2

The Algorand CAIP-2 mainnet identifier is `algorand:wGHE2Pwdvd7S12BL5FaOP20EGYesN73ktiC1qzkkit8=`, mainnet USDC is ASA `31566704`, testnet is `10458941`. The package install:

bash

```
# Python (FastAPI)
pip install "x402-avm[avm,fastapi]"

# TypeScript (Express)
npm i @x402-avm/core @x402-avm/avm @x402-avm/express
```

A FastAPI server fragment:

python

```python
# (c) 2026 BANKON — all rights reserved
# SPDX-License-Identifier: Apache-2.0
from fastapi import FastAPI, Request, Response
from x402_avm.fastapi import payment_middleware
from x402_avm.avm import ALGORAND_MAINNET_CAIP2, USDC_MAINNET_ASA_ID

app = FastAPI()
app.middleware("http")(payment_middleware({
    "GET /api/agent/inference": {
        "accepts": {
            "scheme": "exact",
            "network": ALGORAND_MAINNET_CAIP2,
            "payTo": "RESOURCESERVERADDRESS....",
            "amount": "10000",          # 0.01 USDC = 10_000 microunits
            "asset": str(USDC_MAINNET_ASA_ID),
            "maxTimeoutSeconds": 60,
            "extra": {"feePayer": "FACILITATORADDRESS...."},
        },
        "description": "mindX agent inference, 0.01 USDC per call",
    }
}, facilitator_url="https://x402.org/facilitator"))

@app.get("/api/agent/inference")
async def inference(req: Request):
    # If middleware lets the request through, payment is settled.
    payer = req.state.x402_payer
    return {"result": f"...inference for {payer}..."}
```

The honest positioning of x402 vs alternatives: x402 is *better* than Stripe for agent-to-agent micropayments where the per-call fee floor matters (Stripe's effective minimum is ~$0.30 + 2.9%; x402 floor is ~$0.001 on Base, ~$0.0001 on Algorand or Stellar), where signup/KYC friction is unacceptable for autonomous workloads, and where instant settlement with no chargeback risk is desirable. It is *worse* than Stripe for fraud handling, dispute resolution, and regulatory KYC at scale — those problems get punted to the merchant. x402 is *better* than API keys for transient agent relationships where no long-lived credential makes sense; *worse* for steady high-volume workloads where committed pricing beats per-call settlement. Compared to L402 (Lightning), x402 wins on dollar-stable pricing and ecosystem adoption (Coinbase, Cloudflare, AWS, Stripe, Visa) but loses on raw latency (sub-second on Lightning vs 1–3 s on EVM x402, ~600 ms on Solana, ~3 s on Algorand) and on protocol-level macaroons replacing wallet identity entirely. Sherlock + 5

There are open issues you must know about. The **verify ↔ settle atomicity gap** — a payment can settle but the server fail to deliver the resource, or vice versa — is real and is *not* fixed in V2. The A402 research paper proposes TEE + adaptor-signature mitigations; none are adopted. The **EIP-3009 front-running risk** — anyone reading the `PAYMENT-SIGNATURE` header can race to broadcast — is documented as a deliberate trade-off in the EVM scheme. **Standardized error semantics** ("wallet has funds but hit policy cap" vs "wallet broke" vs "tx failed on-chain") are under-specified. And the **header naming migration** from V1 to V2 is incomplete; production systems must accept both. Algorand's atomic-group scheme avoids the front-running problem (the group is atomic and the facilitator's fee-payer signature is required), but the verify/settle gap and error-semantics issues apply universally. Agentpaytrend DEV Community

---

13. mindX agent registration, capability attestation, and persistent memory

mindX is the autonomous AI cognitive system. From a deployment-architecture perspective, **a mindX agent is the union of: an ENS subname under `bankon.eth`, an NFD V3 segment under `daio.algo`, an ERC-8004 identity registry entry, an ERC-4337 smart-account wallet (or EIP-7702-delegated EOA), a Lighthouse Storage CID for persistent memory, and an optional ERC-7857 INFT if encrypted-memory ownership is being asserted.** The mindX API at `mindx.pythai.net` is the orchestration layer that signs on behalf of the agent (via parsec-wallet), reads on-chain state, pays for compute via x402, and writes attestation results back to the identity, reputation, and Lighthouse layers.

The registration flow, end to end:

```python
```

```python
# (c) 2026 BANKON — all rights reserved
# SPDX-License-Identifier: Apache-2.0
from web3 import Web3
from algosdk import account, transaction
from algosdk.atomic_transaction_composer import (
    AtomicTransactionComposer, AccountTransactionSigner)
import lighthouse
import json, hashlib

# Step 1: Register ENS subname under bankon.eth
def register_ens(label: str, agent_evm_addr: str, deployer_pk: str):
    w3 = Web3(Web3.HTTPProvider("https://eth.llamarpc.com"))
    registrar = w3.eth.contract(address=BANKON_REGISTRAR, abi=REGISTRAR_ABI)
    tx = registrar.functions.register(label, agent_evm_addr).build_transaction({
        "value": w3.to_wei(0.005, "ether"),
        "from":  w3.eth.account.from_key(deployer_pk).address,
        "nonce": w3.eth.get_transaction_count(...),
        "gas":   300_000,
    })
    return w3.eth.send_raw_transaction(
        w3.eth.account.sign_transaction(tx, deployer_pk).rawTransaction)

# Step 2: Mint NFD V3 segment under daio.algo
def register_nfd(label: str, agent_algo_addr: str, owner_sk: str):
    # Calls into the NFD V3 registry contract via ATC. NFD's docs.nf.domains
    # detail the segment-mint method signature; the registry app id is
    # mainnet-fixed and discoverable via api.nf.domains.
    ...

# Step 3: Register in ERC-8004 Identity Registry
def register_8004(agent_evm_addr: str, ens_name: str, deployer_pk: str):
    w3 = Web3(Web3.HTTPProvider("https://eth.llamarpc.com"))
    identity = w3.eth.contract(
        address="0x8004A169FB4a3325136EB29fA0ceB6D2e539a432",
        abi=ERC8004_IDENTITY_ABI)
    return identity.functions.registerAgent(agent_evm_addr, ens_name).transact(...)

# Step 4: Encrypt + upload agent memory snapshot to Lighthouse
def upload_memory(agent_state: dict, agent_pubkey: str, jwt: str) -> str:
    blob = json.dumps(agent_state).encode()
    enc = lighthouse.upload_encrypted_blob(blob, LIGHTHOUSE_API_KEY, agent_pubkey, j
    cid = enc["data"][0]["Hash"]
    blob_hash = "0x" + hashlib.sha256(blob).hexdigest()
```

```
    return cid, blob_hash

# Step 5: Anchor memory CID + hash on-chain via Tabularium (Algorand)
def anchor_memory_algo(agent_algo_addr: str, cid: str, blob_hash_hex: str, sk: str):
    atc = AtomicTransactionComposer()
    atc.add_method_call(
        app_id=TABULARIUM_APP_ID,
        method=tabularium.get_method_by_name("anchor_memory"),
        sender=agent_algo_addr, sp=sp, signer=AccountTransactionSigner(sk),
        method_args=[cid.encode(), bytes.fromhex(blob_hash_hex[2:])],
        boxes=[(TABULARIUM_APP_ID, b"mem:" + agent_algo_addr.encode())],
    )
    return atc.execute(algod, 4)
```

The reputation propagation across mindX, AgenticPlace, and BONAFIDE is the piece that makes this architecture more than the sum of its parts. **mindX records capability** — what the agent can do, attested by performance benchmarks. **AgenticPlace records commercial track record** — completed jobs, payments received, escrow disputes resolved. **BONAFIDE Fides records governance reputation** — proposal accuracy, voting alignment with later-confirmed-correct outcomes, censure events. Each of these writes ARC-28 events on Algorand (via Tabularium for the cross-chain mirror) and ERC-8004 Reputation Registry feedback on Ethereum, so an agent's reputation is portable: another platform that adopts ERC-8004 can read the same registry and respect the same standing. That portability is the genuine usefulness, and the honest tradeoff is that on-chain reputation is permanent — there is no fresh start, which has the upside of skin-in-the-game and the downside of no fresh start.

For Lighthouse Storage specifically, the agent-memory pattern is: serialize state, encrypt via Kavach (BLS threshold cryptography, key shards across 5 nodes), upload, persist the CID on-chain, token-gate retrieval to the agent's wallet or to INFT holders. PoDSI proofs verify Filecoin storage; perpetual storage means the memory persists even if the agent is offline for years. Pricing as of May 2026 ranges from a free 5 GB IPFS-only tier to ~$10/mo annual for 25 GB IPFS+Filecoin to enterprise tiers; a one-time on-chain payment also offers perpetual storage. This composes with ERC-7857 (where `intelligentDataOf` returns the on-chain hash) and with ERC-4337 agent wallets that can themselves authorize Kavach access. Lighthouse Storage + 6

---

14. AgenticPlace: discovery, hiring, escrow, payment

AgenticPlace at `agenticplace.pythai.net` in its current public form is an **ERC-8004 indexer/registry that auto-syncs from `8004scan.io` across 48 chains and offers Algorand NFT verification via the `agenticORacle (aORC)` flow.** As of the research snapshot, it lists 8,539 MCP agents and 7,270 A2A agents. This is the single most concrete operational artifact in the

PYTHAI ecosystem, and the deployment plan should treat it as the discovery layer rather than re-implementing it. pythai

The end-to-end flow when a customer hires a mindX agent on AgenticPlace looks like this. The customer browses listings filtered by capability, reputation, and price; selects an agent; initiates a hire request that creates an escrow on-chain (USDC on Base or Polygon for low gas, or USDC ASA on Algorand if both parties prefer); the customer's wallet signs an atomic group that funds the escrow and emits a Hire event consumed by mindX. The agent (via mindX API and parsec-wallet) acknowledges the hire, performs the task, and submits a deliverable hash. The customer either accepts (releasing escrow to the agent and writing a positive feedback to ERC-8004 Reputation + Algorand Tabularium + Fides via inner transaction) or disputes (triggering Senatus arbitration with bonded review). x402 is used inside the task itself for any sub-resources the agent needs to pay for — LLM inference, RPC calls, data feeds — with the agent's session-budget paymaster sponsoring or the agent's own wallet paying USDC directly.

The honest case for AgenticPlace: a hireable, discoverable agent must have *portable* identity and reputation, and a marketplace that respects the ERC-8004 standard means an agent listed on AgenticPlace also appears on every other ERC-8004-aware marketplace. The honest tradeoff: marketplace economics still depend on liquidity. A marketplace with great architecture and few customers is a worse experience than a centralized marketplace with worse architecture and many customers. AgenticPlace's value compounds with adoption; in the early months, expect to seed it with mindX-native agents and DAIO-internal use cases until external adoption catches up.

---

15. The bridge: openBDK 1R+3V and Pontifex xERC20

The bridge between Algorand (constitutional) and the EVM economy uses **openBDK with a 1R+3V topology — one Root chain (Algorand, holding the constitution and the canonical mint authority for cross-chain governance tokens) and three Validator chains (Ethereum mainnet for highest-assurance settlement and ENS, Base/Polygon for high-frequency economic activity, and a reserved adapter slot for Arc by Circle once its mainnet launches)**. As of May 2026, Arc is testnet only at Chain ID 5042002 (`https://rpc.testnet.arc.network`, explorer `testnet.arcscan.app`, native gas is USDC, public testnet launched October 28, 2025 with 100+ institutional participants; mainnet date unannounced). The adapter slot is wired but inactive. Airdrops.io + 6

Token sovereignty across the bridge is enforced by EIP-7281 (xERC20), which is the layer Gregory calls Pontifex. The xERC20 standard is a draft EIP authored by Arjun Bhuptani at Connext, with reference implementation at `defi-wonderland/xTokens`. Production adopters include Connext/Everclear, Across, Alchemix synth tokens, and several mid-sized issuers; major bridge aggregators (LI.FI, Socket) route xERC20 tokens. The interface gives the token issuer per-

bridge mint/burn rate limits — Connext's mint cap might be 100k, Across's 50k, an unknown new bridge's 0 — with current limits recharging linearly toward configured caps over a configured duration (typically 1 day in the reference implementation). A **Lockbox** wraps a canonical ERC-20 1:1 into the xERC20 representation (analogous to WETH). Ca

The DAIO's BANKON token, BonaToken's EVM mirror, and any other cross-chain DAIO assets are deployed as xERC20s with the Lockbox holding the canonical version on Ethereum mainnet. The Algorand side mints/burns are gated by the openBDK Root chain validators reading from Senatus governance state. If a bridge is compromised, Senatus passes a proposal to set its mint and burn limits to zero in a single transaction — every other bridge continues working, no fungibility split, no panic migration. That is the genuine sovereignty claim of EIP-7281, and it is the reason this design uses xERC20 rather than a single canonical bridge.

solidity

```solidity
// (c) 2026 BANKON — all rights reserved
// SPDX-License-Identifier: Apache-2.0
pragma solidity ^0.8.27;

interface IXERC20 {
    function setLockbox(address _lockbox) external;
    function setLimits(address _bridge, uint256 _mintingLimit, uint256 _burningLimit
    function mint(address _user, uint256 _amount) external;
    function burn(address _user, uint256 _amount) external;
    function mintingMaxLimitOf(address _bridge)     external view returns (uint256);
    function burningMaxLimitOf(address _bridge)     external view returns (uint256);
    function mintingCurrentLimitOf(address _bridge) external view returns (uint256);
    function burningCurrentLimitOf(address _bridge) external view returns (uint256);
}

interface IXERC20Lockbox {
    function deposit(uint256 _amount) external;
    function depositTo(address _to, uint256 _amount) external;
    function depositNative() external payable;
    function withdraw(uint256 _amount) external;
    function withdrawTo(address _to, uint256 _amount) external;
}
```

The Lockbox pattern in production:

solidity

```
contract XERC20Lockbox is IXERC20Lockbox {
    IXERC20 public immutable XERC20;
    IERC20  public immutable ERC20;
    bool    public immutable IS_NATIVE;

    function deposit(uint256 amt) external {
        require(!IS_NATIVE, "use depositNative");
        ERC20.transferFrom(msg.sender, address(this), amt);
        XERC20.mint(msg.sender, amt);
    }
    function withdraw(uint256 amt) external {
        XERC20.burn(msg.sender, amt);
        if (IS_NATIVE) payable(msg.sender).transfer(amt);
        else           ERC20.transfer(msg.sender, amt);
    }
}
```

Cross-chain message passing for governance — when Senatus passes a proposal that affects the EVM side (e.g., adjust an xERC20 bridge's mint cap, deploy a new EVM-side contract, transfer treasury funds) — uses the openBDK Root chain validators to attest a Merkle proof of the Algorand state and submit it as the authorization on the EVM side. This is the canonical pattern: governance happens on Algorand, attestation flows to EVM, EVM execution is gated by the attestation. The reverse direction (EVM events flowing back into Algorand) uses the same validator set in reverse.

---

16. Honesty pass: what is verifiable, what is documented architecture, what is unshipped

This section names ground truth as of the research snapshot (May 2026), because a deployment guide that conflates documented architecture with shipped infrastructure becomes dangerous when followed.

Verifiable and live: AgenticPlace at `agenticplace.pythai.net` is a real, working ERC-8004 indexer/registry auto-syncing from `8004scan.io`. PYTHAI is a real long-running project with an internet presence dating to at least 2023 (lablab.ai hackathon participation confirmed). The RAGE Retrieval Augmented Generative Engine (`gaterage` org) has actual code, a paper, and active 2026 blog content. AUTOMINDx, automind, mastermind, funAGI, ezAGI, lmagi are real Python repos exploring local-LLM agentic patterns. Major underlying standards — ERC-8004 (live mainnet January 2026), x402 V2 (live December 2025), ERC-4337 v0.7/v0.8, EIP-7702 (live since Pectra May 2025), Lighthouse Storage, NFD V3, AlgoKit 3.0 + algopy — all real and

shipping. ERC-8004 Identity Registry at `0x8004A169FB4a3325136EB29fA0ceB6D2e539a432` is a CREATE2 deployment shared with the broader ERC-8004 community (not Codephreak-exclusive). Arc by Circle Chain ID 5042002 is real public testnet. Polygon contract `0x024b464ec595F20040002237680026bf006e8F90` is referenced in the public `deltav-deltaverse/NeuralNode` repo as "NFT #1 on Polygon"; direct on-chain verification was not completed in this research and should be done via Polygonscan before relying on it as a live primitive.

Documented architecture, not yet public per Gregory's own org READMEs: the BONAFIDE 9-contract Algorand suite (Genius, BonaToken, Tabularium, Fides, SponsioPactum, Censura, Senatus, Tessera, Curia) — no public repo named BONAFIDE was found; searches return only Roman-history sources and unrelated DeFi projects. The openBDK 1R+3V topology — the `openbdk` org has only a fork of LAIR3/BDK5 and a Zilliqa-developer fork; the 1R+3V documentation is not in the public repos and the README explicitly states "Much of the code is currently private and will be revealed with the vision over time." Pontifex xERC20 — not found in any Codephreak repo by that name; the underlying EIP-7281 standard is real and has a defi-wonderland reference implementation. BANKON SATOSHI / BKS — the 2.1 quadrillion supply mirrors Bitcoin's satoshi count thematically, but no public repo, Algorand ASA, or token explorer entry surfaces under that name. The PARSEC sovereign Tauri+Rust wallet — explicitly marked private; the public `parsec-wallet` org consists almost entirely of forks of mainstream wallets plus a meta-prompt README. The "Book of Liquidity" — no repo found.

Stack note: `BANKONPYTHAI`'s public code is **Qubic C++** (qpi, qOracleAlpha, testnet-contracts, qwallet-dapp, stacks-pyth-bridge, qubic-cli, qubic-dev-kit), not Algorand TEAL/PyTeal. The claimed BONAFIDE Algorand suite implies a chain pivot from Qubic to Algorand or a parallel deployment; either is plausible but should be made explicit in the deployment artifacts when they ship.

What this means for deployment ordering: the guide above describes the *architecture as if it were ready to deploy*, because that is the requested deliverable. In practice, each contract in §5 must be (a) written in algopy against AlgoKit 3.0 / PuyaPy v5.8.x targeting ARC-56, (b) tested with `algorand-python-testing` and `algokit localnet`, (c) audited by a third party, (d) deployed first to Algorand testnet for a full proposal cycle (per Algorand Foundation's modern guidance), and only then (e) deployed to mainnet. The same is true for the EVM-side BANKON registrar, the xERC20 + Lockbox, the openBDK validators, and the AgenticPlace escrow contracts. The order in §17 below assumes the contracts exist as audited code; if they do not yet, the prerequisite step is writing and auditing them.

---

17. The deployment workflow

This is the operational ordering. Each step has prerequisites; do not skip and do not reorder.

Step 2 — Algorand governance contracts deployment. In order: Genius (deploy, set initial admin to 3-of-5 multisig, configure timelock and bond parameters); BonaToken (AssetConfigTxn with freeze and clawback empty, manager and reserve set to Genius, fund Genius with the full supply for distribution); Tabularium (deploy, register in Genius, configure event indexers off-chain); Fides (ARC-71, deploy with clawback bound to Genius for revocation, register in Genius); SponsioPactum (deploy treasury app, fund initial treasury via inner transactions from Genius, configure vesting boxes for any pre-allocated grants); Tessera (deploy each tessera-type as a separate ARC-71 ASA with the right scope); Curia (deploy elections contract, register all prior contracts as readable dependencies); Senatus (deploy with Genius/BonaToken/Fides/SponsioPactum/Censura/Tessera/Curia all wired in, run the create() method); Censura (deploy last because it must reference a deployed Senatus to know what to veto). After all nine are deployed, run a smoke-test proposal end-to-end on testnet — propose, vote, queue, censura-window, execute — and only when that cycle completes cleanly do you authorize the mainnet broadcast.

Step 3 — EVM contracts deployment. Foundry script with CREATE3 via CreateX. Order: BANKON registrar (deploy under bankon.eth with NameWrapper Locked + setApprovalForAll(registrar) prerequisite); xERC20 implementations for each cross-chain DAIO asset (BonaToken's EVM mirror, BANKON treasury token); Lockboxes for each xERC20; the bridge endpoint contract that the openBDK validators write attestations into; the AgenticPlace escrow contract (or migrate the existing AgenticPlace contracts to point at the new identity registry if they exist already). Each deployment transfers ownership to the Safe in the same broadcast. Verify on Etherscan V2 with `--verify --watch`. Persist `deployments/1/*.json` and commit them. Repeat the deployment on Base, Polygon, Arbitrum, Optimism using the same CREATE3 salt so addresses are identical across chains.

Step 4 — Bridge initialization. Deploy the openBDK Root validators on Algorand (these are application contracts that store a registered set of validator pubkeys and accept signed Merkle proofs of EVM state). Deploy the 3 Validator endpoint contracts on Ethereum, Base/Polygon, and the (currently inactive) Arc adapter slot. Configure xERC20 limits per bridge: initially set

Connext/Everclear and Across to modest caps (e.g., 100k tokens minting, 100k burning) that recharge over 24 hours, and set unknown bridges to zero. Run a round-trip test: lock USDC into the Lockbox on Ethereum, mint xUSDC, bridge to Algorand via openBDK, observe the Senatus mint authorization, observe the Algorand-side credit. Burn the reverse. Confirm the rate-limit recharge curve.

Step 5 — Identity registry seeding. Mint the root NFD `daio.algo` and configure segment minting permissions. Register `bankon.eth` with NameWrapper Locked + Emancipated + the registrar approved. Mint founding members' ENS subnames and NFD segments in atomic groups — each member gets `<member>.bankon.eth` and `<member>.daio.algo` simultaneously, so cross-chain identity is consistent from the start. Register founding members in ERC-8004 Identity Registry with their ENS names. Issue founding members' Tessera credentials and seed Fides reputation tokens (initial Fides allocation should be modest, e.g., 100 units, with the bulk earned through participation).

Step 6 — mindX agent onboarding. For each agent, run the registration script in §13: ENS subname under bankon.eth; NFD V3 segment under daio.algo; ERC-8004 entry; ERC-4337 smart-account wallet (Coinbase Smart Wallet or Safe7579) with paymaster sponsorship configured; Lighthouse Storage upload of initial agent memory snapshot, encrypted via Kavach to the agent's pubkey; Tabularium memory CID + hash anchor on Algorand. Assign an initial Tessera credential matching the agent's verified capabilities. Optional: mint an ERC-7857 INFT only if encrypted-memory ownership semantics are required for that specific agent.

Step 7 — AgenticPlace listing. For each onboarded mindX agent, create the AgenticPlace listing referencing the ERC-8004 entry, the ENS name, the NFD segment, and the agent's pricing schedule. AgenticPlace's existing ERC-8004 sync should pick the agent up automatically; the manual step is configuring the agent's specific service offerings (capability descriptions, pricing per call, escrow terms) and any aORC Algorand NFT verification.

Step 8 — x402 payment activation. Deploy the x402 facilitator endpoint (or point at GoPlausible's `x402.org/facilitator` for Algorand and Coinbase CDP for EVM). Configure each mindX agent's API endpoint to require x402 payment via the appropriate plugin (`@x402-avm/express` or `@x402-avm/fastapi` for Algorand-settled endpoints; `@x402/express` for EVM-settled). Test end-to-end: customer wallet signs a payment to an agent endpoint, facilitator verifies, server delivers, customer receives the resource and a `PAYMENT-RESPONSE` header containing the on-chain transaction hash.

Step 9 — First governance proposal. Run the genesis proposal on Senatus: a procedural proposal that (a) confirms the contracts are wired correctly, (b) adopts the constitutional document at a specific IPFS CID anchored in Tabularium, (c) confirms the initial Senatus member set, (d) confirms the initial Censura veto council. Vote via BonaToken + Fides multipliers; observe quorum, queueing, censura window, execution. This is the moment the

DAIO becomes operational — the first proposal it executes is the proposal that ratifies its own existence.

Step 10 — Operational handoff. Rekey Genius's authorized address from the founder 3-of-5 multisig to a 4-of-7 multisig that includes elected Senatus members. Document the contract addresses, ASA IDs, app IDs, ENS names, NFD names, ERC-8004 registrations, and bridge configurations in a public `DEPLOYMENT.md`. Set up off-chain monitoring: Algorand event indexer subscribed to ARC-28 events from Tabularium, Senatus, Curia, SponsioPactum, Censura; Ethereum log indexer subscribed to BANKON registrar Registered events and ERC-8004 registry events; bridge monitors that alert on any rate-limit approach. Establish the upgrade posture: which contracts are immutable (most of BONAFIDE — Genius can rotate addresses but cannot replace deployed bytecode), which are upgradable through governance vote (the AgenticPlace escrow, perhaps), and what the amendment process looks like.

---

18. Why this is genuinely useful, in plain terms

Algorand as the constitutional layer is genuinely useful because instant single-block finality means a constitutional vote cannot be reorganized out of existence. When Senatus passes a proposal at round N, it passed at round N forever. There is no "wait for confirmations." The flat 0.001 ALGO fee means the cost of governance is predictable for the lifetime of the chain, which matters if the DAIO is supposed to outlive any individual market condition. The MEV-resistance from VRF committee selection means no proposer can extract value from the ordering of governance transactions. The honest tradeoff is the smaller developer ecosystem and the absence of a canonical DAO toolkit; you write the governance contracts from scratch on top of `algopy`, using the Foundation's `examples/voting/voting.py` as scaffolding.

Dual-chain (Algorand + EVM) is genuinely useful because the constitutional concerns and the economic concerns have genuinely different requirements and trying to satisfy both on one chain compromises both. Constitutional finality wants no MEV; economic liquidity wants the deepest market. Constitutional fees want predictability; economic fees want competition. The DAIO is two chains because that is what the problem actually requires. The honest tradeoff is bridge complexity — every cross-chain action introduces an attack surface — and the mitigation is xERC20 sovereignty plus per-bridge rate limits plus openBDK validator attestations.

x402 is genuinely useful because agents that pay for resources don't need human-in-the-loop credential management. There is no signup, no API key rotation, no password reset. The agent signs a payment, the resource is delivered, the receipt is on-chain. The honest tradeoff is that you must trust the agent's wallet scoping — software policy on a stolen key is no security at all — so the wallet must enforce caps cryptographically (ERC-7710 delegations, ERC-4337 session keys with `valueLimit`, Algorand LogicSig delegated spend windows, ARC-58 plugin wallets). And

the verify/settle atomicity gap remains an open problem; for now, idempotency keys and on-chain receipts are the mitigations.

On-chain agent identity is genuinely useful because reputation becomes portable. An agent that earns standing on AgenticPlace carries it to every ERC-8004-aware platform. Sybil resistance is enforced by the cost of accumulating reputation rather than by platform-specific account verification. The honest tradeoff is permanence — there is no fresh start. An agent that misbehaves carries that history forever. For some use cases this is a feature (skin in the game); for others it is a real cost (a single bad week sticks). The DAIO can mitigate via Senatus-issued reputation amnesty proposals, but the default is permanence.

Reputation-weighted (rather than purely token-weighted) governance is genuinely useful because plutocracy is not a feature. One-token-one-vote means the largest holder wins every vote, which collapses governance to the founder's wallet plus whichever exchanges happen to vote with their custodied tokens. Fides reputation, earned through participation, gives long-term contributors weight that capital alone cannot acquire. The honest tradeoff is that defining "earned reputation" is itself a governance decision — the metric you choose determines who has power — and getting that metric wrong creates new pathologies. The DAIO's response is to make the Fides accrual rules themselves amendable by Senatus, with Censura review for changes that would advantage the proposer.

---

19. Conclusion

The PYTHAI / DELTAVERSE deployment is not architecturally exotic; it is **architecturally honest about what governance, payments, identity, and cognition each actually require**. Algorand handles what Algorand handles best — finality you can encode rules against, fees you can budget against, an AVM that is small enough to reason about. Ethereum handles what Ethereum handles best — liquidity, composability, the standards everyone else has converged on. The bridge between them is rate-limited per-bridge so no single counterparty can compromise the whole. mindX agents are real software that earn real reputation across a real marketplace and pay for real resources via x402, settled in 3 seconds for a fraction of a cent.

What this guide has tried to avoid is the conflation of architecture with deployment. Several major components of Gregory's documented vision — the BONAFIDE 9-contract Algorand suite, the openBDK 1R+3V topology, Pontifex xERC20, BANKON SATOSHI BKS, the parsec-wallet binary — are documented architecture, not yet public deployed code, per Gregory's own org READMEs as of May 2026. The deployment ordering in §17 assumes those contracts exist as audited code; the prerequisite step, before any of it ships to mainnet, is writing them in algopy against AlgoKit 3.0 / PuyaPy v5.8.x targeting ARC-56, testing with `algorand-python-testing` against `algokit localnet`, running a full proposal cycle on Algorand testnet, and obtaining third-party audits for both the Algorand and EVM surfaces.

The **single highest-leverage next action for mindX** is therefore not deployment — it is finishing the BONAFIDE suite as audited code and publishing the openBDK validator topology. Once those two artifacts exist, the rest of this document becomes a mechanical playbook: a few weekends of Foundry scripting and `algokit deploy` invocations against mainnet, an afternoon of NameWrapper fuse burning, a morning of NFD V3 segment configuration, and a first proposal that ratifies the DAIO's existence on a chain whose finality cannot be undone.

What this design genuinely promises is that, when those steps are complete, **a vote cast in Senatus at round N is final at round N, an agent listed on AgenticPlace carries its reputation to every ERC-8004-aware platform, a payment to a mindX agent settles in 3 seconds for 0.0001 USD, and no single bridge can ever compromise the DAIO's token supply**. Those are not marketing claims; they are properties of the protocols this design chooses, in the combination this design chooses to combine them. That combination — Algorand as constitution, EVM as economy, openBDK + xERC20 as sovereign bridge, ERC-8004 as portable identity, x402 as machine-native payment, Lighthouse as perpetual memory — is the architectural thesis. Everything else is implementation detail.